

# **DSA Task -04**

***Course Title: Data Structures and Algorithm***

**Course Code:** CS-

**Submitted To:** Mam Ayesha Rashid

**Submitted By:** Muhammad Ali Saagar

**Roll No:** 23021519-095

**Department:** Computer Science

**Class:** BS-III-23-C



**Hafiz Hayat Campus**

**Question no.1:**

## Question No 01 .

```
#include <iostream>
#include <queue>
using namespace std;

queue<int> mergeQueues(queue<int>& queueA, queue<int>& queueB) {
    queue<int> mergedQueue;

    while (!queueA.empty() || !queueB.empty()) {

        if (!queueA.empty()) {
            mergedQueue.push(queueA.front());
            queueA.pop();
        }

        if (!queueB.empty()) {
            mergedQueue.push(queueB.front());
            queueB.pop();
        }
    }
    return mergedQueue;
}

int main() {
    queue<int> queueA, queueB;

    queueA.push(1);
    queueA.push(3);
    queueA.push(5);

    queueB.push(2);
    queueB.push(4);
    queueB.push(6);

    queue<int> mergedQueue = mergeQueues(queueA, queueB);

    cout << "Merged Queue: ";
    while (!mergedQueue.empty()) {
        cout << mergedQueue.front() << " ";
        mergedQueue.pop();
    }

    return 0;
}
```

## Question no. 02

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;

queue<int> reverseQueue(queue<int> queue) {
    stack<int> stack;

    while (!queue.empty()) {
        stack.push(queue.front());
        queue.pop();
    }

    while (!stack.empty()) {
        queue.push(stack.top());
        stack.pop();
    }

    return queue;
}

int main() {
    queue<int> queue;

    queue.push(1);
    queue.push(2);
    queue.push(3);
    queue.push(4);
    queue.push(5);

    queue = reverseQueue(queue);

    cout << "Reversed Queue: ";
    while (!queue.empty()) {
        cout << queue.front() << " ";
        queue.pop();
    }

    return 0;
}
```

## Question No. 03

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;

bool isPalindrome(queue<int> queue) {
    stack<int> stack;
    int size = queue.size();

    for (int i = 0; i < size; i++) {
        stack.push(queue.front());
        queue.pop();
    }

    for (int i = 0; i < size; i++) {
        if (stack.top() != queue.front()) {
            return false;
        }
        stack.pop();
        queue.push(queue.front());
        queue.pop();
    }
    return true;
}

int main() {
    queue<int> queue;

    queue.push(1);
    queue.push(2);
    queue.push(3);
    queue.push(2);
    queue.push(1);

    cout << "Is Palindrome: " << (isPalindrome(queue) ? "True" : "False") << endl;

    return 0;
}
```

## Question No. 04

```
#include <iostream>
#define SIZE 5
using namespace std;

class CircularQueue {
private:
    int queue[SIZE];
    int front, rear;

public:
    CircularQueue() {
        front = rear = -1;
    }

    void enqueue(int value) {
        if ((rear + 1) % SIZE == front) {
            cout << "Queue is full" << endl;
            return;
        }
        if (front == -1) {
            front = 0;
        }
        rear = (rear + 1) % SIZE;
        queue[rear] = value;
    }

    void dequeue() {
        if (front == -1) {
            cout << "Queue is empty" << endl;
            return;
        }
        cout << "Dequeued: " << queue[front] << endl;
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % SIZE;
        }
    }

    void display() {
        if (front == -1) {
            cout << "Queue is empty" << endl;
            return;
        }

        cout << "Queue elements: ";
```

```
        for (int i = front; i != rear; i = (i + 1) % SIZE) {  
            cout << queue[i] << " ";  
        }  
        cout << queue[rear] << endl;  
    }  
};
```

```
int main() {  
    CircularQueue c;  
  
    c.enqueue(10);  
    c.enqueue(20);  
    c.enqueue(30);  
  
    c.display();  
  
    c.dequeue();  
    c.display();  
  
    return 0;  
}
```